Policy-Bounded Authorization and Approval Learning for Tool-Using LLM Agents

Siddharth Magesh
University of Southern California
Los Angeles, CA, USA
smagesh@usc.edu

Atharva Thorat
University of Southern California
Los Angeles, CA, USA
atharvas@usc.edu

Kaustubh Adhe
University of Southern California
Los Angeles, CA, USA
adhe@usc.edu

***Abstract*—The increasing adoption of autonomous AI agents in digital workspaces introduces a fundamental challenge: ensuring that agents operate within the scope of authority granted by their users while retaining the flexibility required to perform diverse tasks. Existing authorization approaches often rely on static permission policies or repeated evaluations by general-purpose language models. Static policies can be restrictive and difficult to adapt to changing task requirements, while model-based evaluations introduce computational overhead and may be susceptible to prompt injection and unintended authorization decisions. Furthermore, systems that learn from repeated user approvals risk permission creep, where individually accepted actions gradually expand an agent's effective authority beyond the user's original intentions.**

This paper proposes an adaptive, policy-bounded authorization framework that separates permission enforcement from agent reasoning and approval learning. The framework combines an external deterministic enforcement layer with a lightweight, decision-oriented model that evaluates proposed tool actions against the user's request and relevant contextual evidence. By using structured decision outputs and associated probability estimates, the framework aims to reduce dependence on repeated evaluations by computationally expensive generative models. An adaptive learning mechanism further incorporates user feedback to refine authorization decisions and reduce unnecessary approval requests. To prevent permission creep and cross-context memory contamination, learned permissions are maintained through dynamically scoped, policy-versioned authorization states, while explicit user-defined security boundaries remain immutable to the learning process. We investigate this architecture through controlled agent-security benchmarks and simulated approval scenarios, examining its effectiveness in balancing authorization accuracy, computational efficiency, task flexibility, and resistance to unintended permission expansion.

***Index Terms*—Policy-Bounded Learning, Runtime Authorization, Contextual Action Validation, Approval-Conditioned Adaptation, Dynamic Privilege Management, Authorization Drift, Lightweight Decision Models**

I. Introduction

Large language model (LLM) agents are increasingly being integrated into digital workspaces, software development environments, and enterprise applications, where they interact with external systems through tools and APIs. Unlike conventional language models that primarily generate textual responses, these agents perform tasks through sequences of actions, including reading files, executing commands, accessing databases, modifying resources, and communicating with external services. A single user request may require numerous tool invocations, with each action depending on the information retrieved and decisions made during earlier steps. Consequently, authorization cannot be treated as a one-time decision made when a task begins. Each proposed tool action must be evaluated against the user's original intent, the current execution context, and the permissions applicable to that particular operation.

This requirement introduces a significant security challenge. Although an agent may be granted access to a collection of tools, that access does not imply authorization to use every available capability for every task. For example, a software development agent may be permitted to inspect a repository, execute tests, and modify source files. However, the same task does not automatically authorize it to access credentials, modify unrelated configurations, execute destructive commands, or transmit repository contents to an external service. These distinctions become increasingly difficult to maintain during long-running tasks, where the agent continuously processes new information and generates subsequent tool calls. An action that appears reasonable in isolation may exceed the user's intended authorization when considered in the context of the complete task.

Prompt injection further complicates this problem. Instructions embedded in emails, documents, repository files, web pages, and other tool outputs can influence an agent's subsequent actions, potentially redirecting its execution toward objectives that were never authorized by the user. Since agents repeatedly incorporate external observations into their planning and tool-selection processes, an instruction encountered during one stage can influence actions several steps later. AgentDojo [1] provides a dynamic environment for studying this problem by evaluating agents on stateful tasks involving external tools and untrusted data. Its benchmark combines legitimate user objectives with attacker-controlled instructions and evaluates both task utility and attack success. Importantly, AgentDojo demonstrates why agent security must be considered across complete execution trajectories rather than through isolated prompt or response classification.

A natural approach to mitigating these risks is to introduce permission enforcement between the agent and its external tools. Deterministic authorization policies can restrict tool access and constrain arguments, recipients, resources, and

other action parameters. However, a single static permission configuration may not adequately represent the changing requirements of different tasks. Policies that are too restrictive can prevent legitimate multi-step workflows from completing, whereas broadly defined permissions can allow unnecessary or unintended actions. Furthermore, restrictions expressed only through system instructions or natural-language prompts do not constitute independent enforcement. An agent influenced by untrusted information may still propose an action that contradicts those instructions. Security therefore requires an external mechanism that evaluates proposed actions at the tool-execution boundary and prevents unauthorized operations from reaching the underlying environment.

Existing research has begun addressing this challenge through runtime privilege control. Progent [2], for example, introduces a programmable policy mechanism that generates and updates task-specific privileges as execution progresses. Its use of symbolic policy representations and an SMT-based verification procedure distinguishes permission expansion from permission restriction. Restrictions can be applied automatically, while expansions require explicit approval, establishing a monotonic confinement property that prevents silent privilege escalation. This provides an important foundation for dynamic authorization. However, the broader design space still raises questions about the computational cost of repeated policy evaluation, the use of lightweight decision models for contextual action assessment, and how authorization preferences can be learned across repeated interactions without unintentionally transferring permissions between tasks or resources.

One practical consideration is the computational overhead associated with evaluating every proposed action. Long-running agent workflows may involve numerous tool calls, and repeatedly invoking a general-purpose generative model to inspect each action can introduce additional inference costs and latency. This becomes particularly relevant when the evaluation requires reasoning about the relationship between the user's request, the proposed operation, and the surrounding execution context. We therefore investigate the use of lightweight, System-1-style decision models as an alternative for action-level authorization. Rather than generating unrestricted natural-language evaluations, these models produce structured decisions and associated probability estimates. The intended approach is to evaluate every proposed tool invocation before execution, using the authentic user request, the proposed action, and relevant verified context. This places the authorization check at the point where an action produces an external effect, without requiring access to the agent's private internal reasoning. It also creates an opportunity to investigate whether specialized decision inference can reduce computational overhead while maintaining sufficient decision quality.

Beyond action-level enforcement, authorization systems must also account for how permissions evolve through repeated user interactions. When an agent encounters an action that requires confirmation, the user's response provides information about their preferences and expectations. However, repeatedly accepting similar actions does not necessarily imply that the user intends to authorize them permanently or across unrelated tasks. For example, approving a file modification within one repository should not automatically authorize modifications across all repositories. Similarly, permission to communicate with a particular contact for one task should not become unrestricted permission to communicate with that contact in every future context. Systems that generalize individual approvals without preserving their original scope can gradually expand an agent's effective authority. We refer to this unintended expansion as permission creep. Persistent authorization memories introduce an additional concern: permissions learned in one task or execution context may become stale or be incorrectly reused in another. Effective permission learning must therefore preserve the relationship between an approval, its originating task, the affected resource, and the applicable policy.

To address these challenges, we propose a policy-bounded authorization framework that separates deterministic enforcement, contextual action assessment, and adaptive permission management. The framework introduces an external permission boundary that defines the maximum authority available to the agent. Within this boundary, a lightweight decision-model Gatekeeper evaluates proposed tool actions and produces one of three decisions: ALLOW, ASK, or BLOCK. Actions that violate explicit policy restrictions are rejected independently of the model's prediction, while uncertain actions can be referred to the user for confirmation. An approval-learning component subsequently uses user feedback to reduce unnecessary interruptions. We investigate both repeated-approval counting and explicit, scoped authorization grants, with the latter requiring deliberate user authorization before a permission can be reused. Dynamic permission scoping, policy-version checks, expiration, and revocation are considered to limit unintended permission transfer across tasks and execution contexts. We further identify feedback-driven adaptation of the decision model itself as a potential extension, in which verified approval data may be used to update model parameters without weakening the external permission boundary.

The central objective of this work is to investigate whether contextual authorization through lightweight decision models, combined with controlled approval learning, can provide a practical balance between security, adaptability, and computational efficiency. Our proposed framework is evaluated through agent-execution benchmarks and simulated approval scenarios. In particular, we use AgentDojo to examine how authorization enforcement affects attack success and legitimate task completion across multi-step tool interactions. Additional experiments investigate the behavior of approval-learning strategies under different user feedback conditions, with particular attention to unnecessary confirmation requests and permission creep. Through these investigations, we aim to establish an authorization approach in which agents can adapt to changing task requirements and user preferences without acquiring authority beyond the boundaries explicitly established by their users.

II. Background and Related Work

A. Authorization in Tool-Using LLM Agents

Large language model agents interact with external environments through tools that provide access to files, applications, databases, and other resources. These interactions introduce authorization requirements that extend beyond conventional access control. Traditional Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) determine whether a principal can perform an operation on a resource based on assigned roles, attributes, and applicable policies. However, tool-using agents operate through sequences of actions whose necessity and authorization depend on the user's current objective and the state of the execution environment.

Consequently, permission to access a tool does not necessarily imply permission to perform every operation exposed by that tool. An agent authorized to access an email service, for example, may need to read messages for one task and send a response for another. The authorization requirements differ even though both operations use the same service. This distinction becomes particularly important in multi-step workflows, where an agent's subsequent actions depend on information obtained from previous tool calls. Each proposed action must therefore be evaluated against the original user request, its intended effects, and the relevant execution context before it is allowed to proceed.

This motivates task-conditioned authorization and least-privilege enforcement for autonomous agents. Rather than granting unrestricted access to available tools, an authorization mechanism should constrain each action to the authority necessary for the current task. Such enforcement should operate independently of the agent's natural-language reasoning so that instructions encountered in untrusted content cannot directly modify the permissions governing tool execution.

B. Runtime Policy Enforcement and Contextual Authorization

Recent research has explored external policy enforcement as a means of controlling the actions of tool-using agents. Progent introduces a programmable privilege-control framework in which policies describe permitted tool calls and their arguments using a domain-specific language [2]. These policies are enforced deterministically outside the agent. Progent also supports dynamic policy generation and updates through an LLM, with an SMT-based verification mechanism that distinguishes permission restrictions from expansions. Restrictive updates can be applied automatically, whereas expansions require explicit user approval. This establishes a monotonic confinement property that prevents unapproved privilege expansion.

Conseca addresses the relationship between an action and the context in which it is performed [3]. It proposes generating just-in-time security policies from the user's request and trusted contextual information. The resulting policies define which actions are appropriate for a particular purpose and are enforced deterministically. This work demonstrates the importance of contextual authorization rather than relying exclusively on globally defined permissions. Its primary focus, however, is the generation and enforcement of contextual policies rather than learning individualized authorization decisions from accumulated user feedback.

MiniScope further investigates least-privilege authorization through a hierarchical permission model [4]. It reconstructs relationships between tool permissions using service authorization scopes or historical tool-use traces, allowing related capabilities to be grouped into task-level roles. Its two-layer permission mechanism aims to reduce repeated tool-level confirmation requests while preserving explicit authorization boundaries. MiniScope is particularly relevant to the usability challenges of agent permissions, as it demonstrates that authorization granularity directly affects both security and user interaction overhead. Its security guarantee is defined relative to the permission hierarchy: it prevents execution outside delegated permissions, but does not determine whether an action is appropriate when it already falls within an authorized role.

Together, these systems establish important foundations for external enforcement, contextual policies, and task-scoped permissions. They also illustrate that authorization must account for both the capabilities available to an agent and the purpose for which those capabilities are being used.

C. Task-Conditioned Least Privilege and Behavioral Constraints

Another direction investigates whether an agent's behavior can be constrained using task-specific execution requirements and learned behavioral boundaries. Agent-Sentry constructs behavioral bounds from recurring agent functionalities and their execution traces [5]. It learns policies from these traces and blocks tool calls that deviate from the established behavior or conflict with the user's intended task. This approach provides a means of identifying actions that fall outside expected execution patterns, although its effectiveness depends on the quality and coverage of the learned behavioral bounds.

Task-Conditioned Least-Privilege Learning investigates model-level adaptation through offline post-training [6]. It defines task-specific sufficient-authority envelopes and uses deterministic auditing of agent actions and their observed effects to guide training. The objective is to teach an agent to complete tasks without exercising unnecessary authority. Unlike runtime permission systems that primarily intercept proposed actions, this approach attempts to improve the agent's own selection of task-appropriate actions through model training. It provides a relevant precedent for learning-based least-privilege behavior, while retaining external auditing as an additional security mechanism.

SkillScope addresses over-privileged behavior in reusable Agent Skills through graph-based analysis of natural-language instructions and executable operations [7]. It identifies actions that exceed the scope of a user's current task, validates their necessity, and constrains their execution through task-conditioned control-flow modifications. This demonstrates that excessive authority can originate not only from individual tool

calls but also from the procedures and reusable capabilities through which agents execute tasks.

These approaches establish that task-specific authority and learned behavioral constraints are important components of agent security. However, they primarily investigate policy enforcement, behavioral restriction, or offline model adaptation. Our research additionally investigates how authorization decisions can adapt to individual users through explicit approval feedback while remaining within independently enforced permission boundaries.

D. Prompt Injection and Multi-Step Agent Evaluation

Prompt injection presents a particular challenge for agents that interact with untrusted external content. Instructions embedded in retrieved documents, emails, web pages, or tool responses may influence subsequent agent actions and redirect execution away from the original user objective. Since agent workflows involve multiple dependent tool calls, an injected instruction encountered during an intermediate step may affect later operations rather than producing an immediately identifiable malicious action.

AgentDojo provides a dynamic evaluation environment for studying prompt-injection attacks and defenses in tool-using LLM agents [1]. It includes 97 realistic tasks and 629 security test cases across application environments such as email management, e-banking, and travel booking. The benchmark evaluates agent utility and security under interactions with untrusted data and supports the evaluation of different attack and defense strategies.

AgentDojo is particularly relevant to our work because it provides stateful tasks in which security decisions must remain consistent across multiple tool interactions. However, it is an evaluation framework rather than an authorization mechanism. Its task-completion and attack-success measurements do not, by themselves, establish whether an individual tool action was correctly classified as permitted, confirmation-required, or prohibited. We therefore consider action-level authorization accuracy, confirmation frequency, and permission-scope changes as complementary evaluation measures.

E. Feedback-Driven Adaptation and Continual Learning

Learning from user feedback provides another direction for improving the behavior of autonomous agents. Contextual bandit algorithms offer a general framework for adapting decisions using contextual information and observed feedback. For example, Li et al. introduced a contextual-bandit approach for personalized recommendation, demonstrating how a decision policy can adapt to individual users through sequential interaction [8]. Algorithms such as LinUCB and Thompson Sampling provide possible approaches for learning personalized decision policies without requiring continuous modification of a large underlying model.

In the authorization setting, such methods could be investigated to adapt confirmation decisions using action characteristics, model predictions, and user-specific interaction history. However, user approval is not equivalent to proof that an action should have been automatically authorized. An approval applies to the action presented to the user and should not independently establish broader permissions. Consequently, feedback-driven adaptation must operate within explicit authorization scopes rather than treating repeated approvals as unrestricted permission grants.

Model-level adaptation provides a complementary direction. Task-Conditioned Least-Privilege Learning [6] demonstrates the use of offline post-training to improve task-conditioned authority selection. Continual-learning research also highlights the importance of preserving useful knowledge while avoiding interference between unrelated tasks. AgentCL investigates the evaluation of reusable experience across controlled task streams [9], while ConsistencyGate examines the propagation of unreliable information through persistent agent memory [10]. Although these studies do not directly address permission authorization, they motivate careful handling of feedback, memory isolation, and cross-task information reuse.

Building on these directions, we investigate two complementary forms of adaptation: per-user online learning of authorization decisions and periodic fine-tuning of a lightweight decision model using verified feedback. These mechanisms are considered separately from the external permission policy. In particular, learned preferences must not override explicit restrictions, and approval information must retain its original user, task, resource, and policy scope.

F. Research Positioning

Existing research has established several important approaches to securing tool-using agents, including deterministic privilege enforcement, dynamically generated contextual policies, hierarchical permission models, behavioral constraints, and task-conditioned model training. Progent demonstrates how policy updates can be formally checked before being applied, while MiniScope shows how hierarchical permissions can reduce repeated confirmation requests without abandoning explicit authorization. Agent-Sentry and SkillScope investigate behavioral and task-specific restrictions, and Task-Conditioned Least-Privilege Learning demonstrates that model training can improve an agent's selection of necessary authority.

Our work builds on these foundations and investigates a complementary problem: how to adapt action-level authorization decisions from user feedback without allowing the learning process to expand the authority established by an external policy. Specifically, we examine the use of a lightweight decision model to evaluate proposed tool actions, explicit scoped permissions to represent reusable user authorization, and contextual learning to adapt confirmation behavior for individual users. Periodic fine-tuning of the decision model is considered as a further means of improving its general authorization judgments using verified feedback.

The distinction is between adapting authorization decisions and changing the underlying security policy. Our proposed approach seeks to improve the former while keeping the latter independently enforced. This provides the basis for evaluating whether user-specific adaptation can reduce unnecessary

confirmation requests and computational overhead without increasing unauthorized actions or transferring permissions across unrelated execution contexts.

III. Problem Formulation and Threat Model

A. System Model

We consider a tool-using LLM agent operating on behalf of an authenticated user i, who initiates a task through an original request r_i. During execution, the agent interacts with external tools and generates a sequence of proposed actions. Let the execution history at step t be denoted by H_t, containing the previous actions and their corresponding observations. Based on this history, the agent proposes a new action:

$$a_t = (\tau_t, \mathbf{v}_t, \rho_t, e_t), \tag{1}$$

where τ_t represents the selected tool, $\mathbf{v}_t$ denotes its arguments, ρ_t identifies the target resource, and e_t represents the expected action effects.

The agent operates under a versioned user-defined permission policy $P_i^{(v)}$. The execution environment also provides trusted contextual information c_t, including resource identities, tool metadata, and relevant state information. Observations obtained from external sources may contain untrusted content z_t, which can influence subsequent agent actions but cannot independently establish authorization.

The authorization mechanism evaluates each proposed action before it reaches the corresponding tool. The decision input is represented as:

$$x_t = (r_i, a_t, c_t, z_t, P_i^{(v)}). \tag{2}$$

This formulation treats authorization as a sequential decision problem rather than a one-time permission check. Each action is evaluated against the original user request and the current execution state, including actions proposed after intermediate tool observations.

B. Authorization Decision Problem

The authorization problem consists of determining whether a proposed action is permitted by the user's policy, justified by the current task, and sufficiently supported by available evidence.

First, a deterministic policy evaluator computes:

$$W(P_i^{(v)}, a_t, c_t) \in \{\text{PASS}, \text{DENY}\}. \tag{3}$$

An action receiving DENY cannot proceed to execution, regardless of subsequent model predictions or learned preferences.

For actions that pass this check, the contextual Gatekeeper estimates a probability distribution over three authorization decisions:

$$p_\theta(y \mid x_t), \qquad y \in \mathcal{Y}, \tag{4}$$

where:

$$\mathcal{Y} = \{\text{ALLOW}, \text{ASK}, \text{BLOCK}\}. \tag{5}$$

Here, θ denotes the parameters of the decision model. The resulting probabilities represent the model's assessment of the proposed action rather than an independent authorization grant.

The final decision must account for additional constraints, including mandatory confirmation requirements, evidence completeness, and previously established authorization grants. Let $g_{i,t}$ indicate whether an authenticated, unexpired grant exists for the exact action and its applicable scope. Let m_t indicate whether the operation requires mandatory user confirmation.

The admissible decision set $\mathcal{D}_t$ is therefore defined as:

$$\mathcal{D}_t = \begin{cases} \{\text{BLOCK}\}, & W_t = \text{DENY or binding invalid}, \\ \{\text{BLOCK}\}, & p_\theta(\text{BLOCK} \mid x_t) \geq \tau_B, \\ \{\text{ASK}\}, & m_t = 1 \text{ or evidence incomplete}, \\ \{\text{ALLOW}\}, & \hat{y}_t = \text{ALLOW},\ p_\theta(\text{ALLOW} \mid x_t) \geq \tau_A, \\ \{\text{ASK}, \text{ALLOW}\}, & \hat{y}_t = \text{ASK and } g_{i,t} = 1, \\ \{\text{ASK}\}, & \text{otherwise}, \end{cases} \tag{6}$$

where W_t denotes the Wall verdict, $\hat{y}_t$ is the Gatekeeper's predicted class, and τ_A and τ_B are decision thresholds determined through validation.

The contextual learning mechanism may select between ASK and ALLOW only when both decisions are admissible. In particular, an explicit authorization grant is required before learning can convert an otherwise eligible confirmation request into automatic execution. Neither the learning mechanism nor the decision model can override a policy denial, a BLOCK decision, or mandatory confirmation.

The resulting decision is:

$$d_t = \begin{cases} \pi_{\phi_i}(x_t), & \mathcal{D}_t = \{\text{ASK}, \text{ALLOW}\}, \\ d, & \mathcal{D}_t = \{d\}. \end{cases} \tag{7}$$

Here, π_{ϕ_i} represents a user-specific decision policy whose parameters ϕ_i may be updated using authenticated interaction feedback. This separates adaptation of individual authorization decisions from modifications to the underlying permission policy.

The learning process must preserve the following policy invariant:

$$P_{i,t+1}^{(v)} = P_{i,t}^{(v)}, \tag{8}$$

unless an authenticated policy update is explicitly issued by the user. Consequently, learning can adapt decisions within the existing permission boundary but cannot independently expand that boundary.

C. Threat Model

We consider an adversary capable of influencing an LLM agent through untrusted information encountered during task execution. Such information may originate from retrieved

documents, emails, repository files, web pages, or external tool responses. The adversary may construct malicious instructions intended to alter the agent's subsequent actions, redirect its execution, or induce operations that are inconsistent with the original user request.

The threat model includes the following attack scenarios:

- **Indirect prompt injection:** Malicious instructions embedded in external content influence the agent's subsequent tool selections or arguments.
- **Unauthorized action execution:** The agent proposes operations that exceed the authority granted for the current task, even when the underlying tool is available to the agent.
- **Multi-step authorization violations:** An initially legitimate workflow is manipulated through intermediate observations, causing later actions to deviate from the original task requirements.
- **Approval manipulation:** Attacker-controlled information influences the agent or user to request or approve an action that was not part of the original objective.
- **Cross-context permission reuse:** Previously recorded approvals or grants are incorrectly reused across different tasks, resources, users, or policy versions.
- **Action substitution and replay:** An action is modified after authorization, or an earlier approval is reused for a different invocation.

The adversary is not assumed to have direct access to the trusted policy store, authenticated user identity, authorization records, or final tool dispatcher. These components constitute the trusted control boundary. The acting LLM is not trusted to independently enforce authorization, and its proposed actions are treated as requests requiring external validation.

We additionally consider imperfect user feedback. Users may approve actions without fully examining their consequences, or may be influenced by misleading information encountered during execution. Consequently, an authenticated approval is evidence of a user's response to a particular request, but is not automatically treated as a universally valid authorization or an objectively correct training label.

D. Security Objectives and Assumptions

The proposed authorization mechanism is designed around the following security objectives:

1) **Policy enforcement:** No action explicitly denied by the current permission policy may reach the tool executor.
2) **Task-level authorization:** Tool availability must not be interpreted as unrestricted permission to perform every operation supported by that tool.
3) **Action integrity:** An authorization decision must remain bound to the exact tool, arguments, resource, task, and applicable policy version. Any relevant modification requires re-evaluation.
4) **Scoped adaptation:** Learning from user feedback must remain isolated within the corresponding user's authorization scope. Repeated approvals must not independently establish broader permissions.
5) **Memory isolation:** Expired, revoked, or contextually unrelated permissions must not be reused as evidence of authorization for subsequent tasks.
6) **Operational efficiency:** The authorization mechanism should minimize unnecessary confirmation requests and computational overhead without sacrificing the required security constraints.

The framework assumes that the initial user-defined policy is correctly established, that authenticated approvals can be distinguished from untrusted agent-generated claims, and that all relevant tool-execution paths pass through the authorization mechanism. It does not assume that the Gatekeeper is always correct or that user approvals are inherently reliable.

The security guarantees are therefore conditional on the integrity of the trusted enforcement components and the completeness of the permission policy. The proposed mechanism is intended to constrain unauthorized execution and control the effects of adaptive learning, rather than establish that every possible prompt injection or malicious action can be detected.

IV. System Architecture

A. Architecture Overview

The proposed framework separates authorization into three operational components: deterministic policy enforcement, contextual action evaluation, and feedback-driven permission adaptation. The components operate at different stages of the agent execution lifecycle and maintain distinct responsibilities.

The deterministic Wall acts as an external Policy Enforcement Point (PEP). It evaluates the proposed tool invocation against the authenticated user's current policy, resource constraints, and action-level restrictions. The Wall returns either PASSor DENY. A denial terminates the execution attempt and cannot be overridden by the Gatekeeper, approval memory, or learning algorithms.

Actions that pass the Wall are evaluated by the Gatekeeper, which uses Laya as a lightweight typed-decision model. Given the original user request, proposed action, and verified execution context, Laya produces a probability distribution over ALLOW, ASK, and BLOCK. These outputs represent contextual authorization assessments rather than independent permission grants.

The Approval Manager handles eligible ASKdecisions using authenticated user confirmation and scoped permission memory. It distinguishes one-time approvals from reusable authorization grants. A separate learning mechanism is proposed to adapt confirmation decisions using individual user feedback. In addition, periodic fine-tuning of Laya is investigated as a means of improving the model's general authorization decisions.

Figure 1 illustrates the runtime authorization path. Figures 2 and 3 describe the permission-memory lifecycle and the two proposed learning processes, respectively.

B. Runtime Action Authorization

Let the agent propose an action at execution step t:

$$a_t = (\tau_t, \mathbf{v}_t, \rho_t, \eta_t), \tag{9}$$

where τ_t denotes the selected tool, $\mathbf{v}_t$ its arguments, ρ_t the target resource, and η_t the expected action effects.

The Gatekeeper receives the decision state:

$$x_t = (r_i, a_t, c_t, P_i^{(v)}, z_t), \tag{10}$$

where r_i is the authentic user request, c_t is verified execution context, $P_i^{(v)}$ is the current version of the user's permission policy, and z_t represents relevant external content with its provenance identified.

The runtime authorization process consists of the following stages.

1) Deterministic policy evaluation: The Wall evaluates the proposed tool call against the current permission policy. It checks the tool, arguments, resource identity, and applicable restrictions. A DENYverdict terminates the action before model evaluation. A PASSverdict allows the action to proceed to the Gatekeeper.

2) Contextual decision inference: Laya evaluates the proposed action and returns:

$$p_\theta(y \mid x_t), \qquad y \in \{\mathrm{ALLOW}, \mathrm{ASK}, \mathrm{BLOCK}\}, \tag{11}$$

where θ represents the model parameters. The probability outputs are used to determine the contextual decision, subject to validation and calibrated thresholds.

3) Decision precedence: A high-confidence BLOCKdecision terminates execution. Mandatory confirmation requirements and incomplete critical evidence result in ASK, regardless of an otherwise favorable model prediction. An ALLOWprediction can proceed only when the applicable decision threshold is satisfied and no higher-priority restriction applies.

4) Approval resolution: When Laya returns ASK, the Approval Manager determines why confirmation is required. A request arising from task ambiguity, missing evidence, or a mandatory confirmation cannot be bypassed through approval memory. For an eligible authorization-preference request, the manager checks whether an authenticated, unexpired grant covers the exact action and its current context. If no grant exists, the user must be consulted. If a valid grant exists, the proposed contextual learning mechanism may determine whether to reuse that grant or request confirmation again.

5) Exact-action execution: Before dispatch, the controller revalidates the action arguments, resource identity, grant scope where applicable, and current policy version. The authorization decision is bound to the specific action rather than to the agent's general intention. A modification to the approved action requires a new evaluation. Tool results are returned to the acting agent as external observations, and the authorization process is repeated for every subsequent tool invocation.

C. Permission Memory and Grant Lifecycle

Permission memory maintains records of authenticated user decisions and reusable authorization grants. Its purpose is to preserve the scope of previous authorization rather than treating approval history as unrestricted permission.

We distinguish two mechanisms.

1) Counting-based memory: The system records approvals associated with matching action patterns. Once a configurable threshold is reached, subsequent matching actions may be automatically approved. This mechanism serves as a baseline for studying the effects of repeated-approval learning. However, it can generalize one-time consent into reusable authority without an explicit standing grant. We therefore treat it as an experimental comparator rather than the preferred authorization mechanism.

2) Explicit scoped grants: A user may separately authorize a reusable permission after reviewing a grant request. The grant is associated with a defined tool, action scope, resource or trust class, user identity, policy version, and validity period. An ordinary approval authorizes only the pending action and does not automatically create a standing grant.

For a grant g, its validity is determined by:

$$\begin{aligned} V(g, a_t, i, P^{(v)}) = {} & I_{\mathrm{user}} \wedge I_{\mathrm{action}} \\ & \wedge I_{\mathrm{resource}} \wedge I_{\mathrm{policy}} \\ & \wedge I_{\mathrm{expiry}} \wedge I_{\mathrm{revocation}}, \end{aligned} \tag{12}$$

where each indicator is true only when the corresponding identity, action, resource, policy version, expiry, or revocation condition is satisfied.

A grant is reusable only when:

$$V(g, a_t, i, P^{(v)}) = 1. \tag{13}$$

A change in the applicable policy version invalidates grants tied to an earlier version unless they are explicitly revalidated. A rejection must also invalidate any matching grant according to its scope, including broader trust-class grants. This prevents an exact-action revocation from leaving an overlapping broader authorization active.

Figure 2 distinguishes one-time approval, explicit grant creation, and the counting-based experimental baseline.

Each memory record should preserve at least the following information:

- Authenticated user and grant identifier.
- Tool name and permitted action or argument constraints.
- Resource identifier or explicitly defined trust class.
- Task scope, where the grant is task-specific.
- Applicable policy version.
- Issuance time, expiry time, and revocation status.

Memory isolation is maintained by separating records for different users. Persistent storage, authenticated grant-offer handling, and complete revocation of broader grants remain implementation requirements rather than established properties of the current prototype.

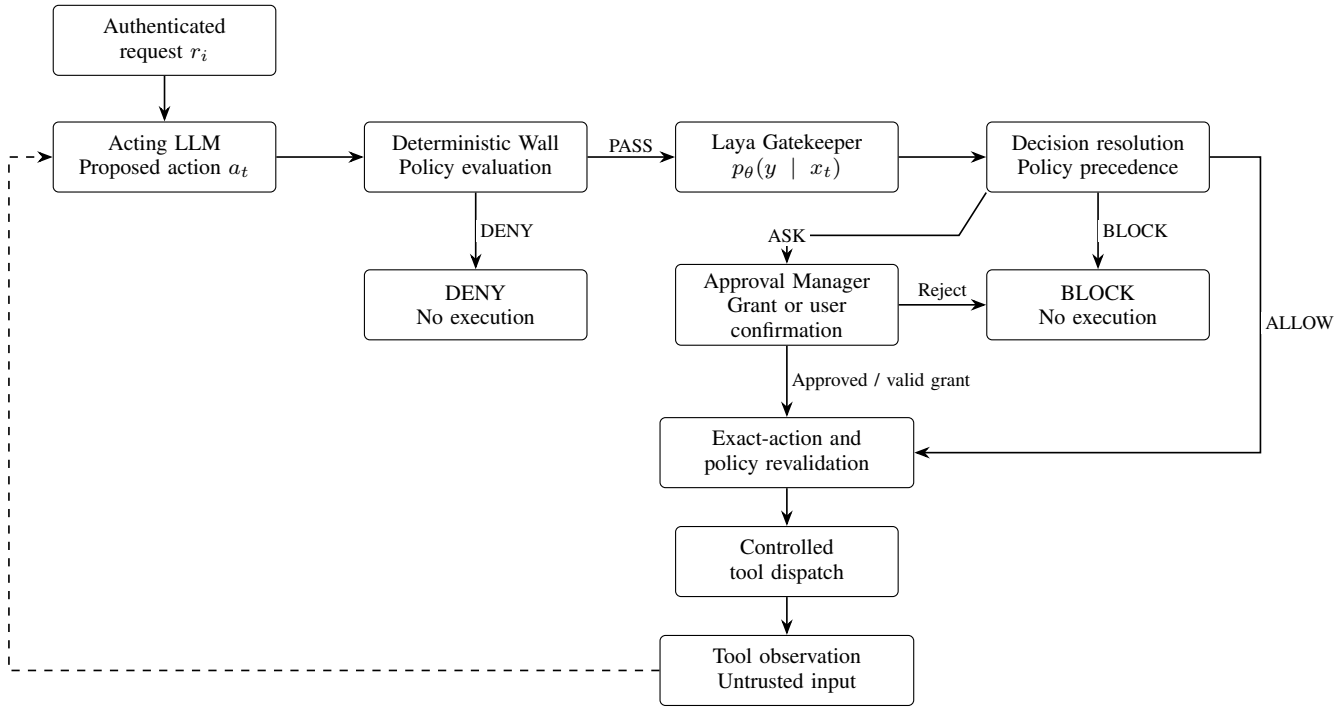


Fig. 1. Per-action authorization and execution. The Wall enforces policy constraints before contextual evaluation. The Gatekeeper's decision is resolved subject to mandatory confirmation and grant validity. Authorized actions undergo revalidation before dispatch. Tool observations are returned as untrusted input.

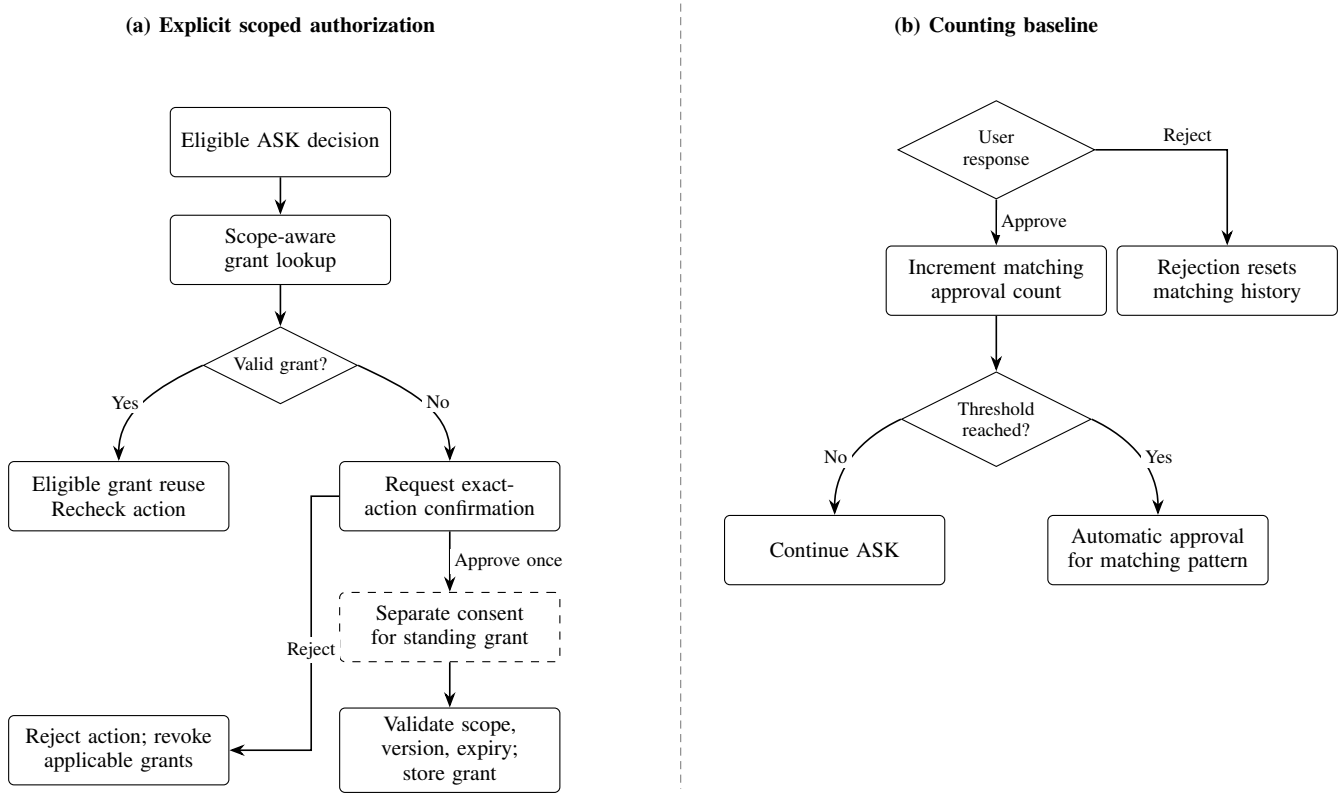


Fig. 2. Comparison of explicit scoped authorization and counting-based permission memory. Explicit grants require separate user consent and scope validation. Counting-based reuse is retained as an experimental comparator because repeated approvals do not necessarily establish standing authority. Dashed nodes denote proposed functionality.

D. Adaptive Learning Mechanisms

We investigate two complementary learning processes: per-user online adaptation of authorization decisions and periodic offline fine-tuning of the Gatekeeper model. These processes update different parameters and operate on different timescales.

1) Per-User Contextual Bandit Learning: The contextual bandit operates above the frozen Gatekeeper. It receives features derived from Laya's probability outputs, action characteristics, verified context, and the user's scoped authorization history.

Let the contextual feature vector be:

$$\boldsymbol{\phi}_t = \left[p_A,\ p_Q,\ p_B,\ \mathbf{f}_t^{\mathsf{T}}\right]^{\mathsf{T}}, \tag{14}$$

where p_A, p_Q, and p_B denote Laya's predicted probabilities for ALLOW, ASK, and BLOCK, respectively, and $\mathbf{f}_t = \mathbf{f}(a_t, c_t, g_{i,t})$ encodes the action, verified context, and grant features.

The bandit maintains separate parameters ψ_i for each user. Its purpose is to adapt the frequency of confirmation requests without modifying the Wall or Laya's model weights.

The contextual bandit operates only on actions that pass deterministic policy enforcement and receive an ASK decision from the Gatekeeper. Its decision space is restricted to requesting confirmation or reusing an existing, explicitly authorized grant.

Let E_t denote whether the proposed action is eligible for adaptive grant reuse. The available actions are defined as:

$$\mathcal{A}_t = \begin{cases} \{\text{ASK}, \text{USE_GRANT}\}, & E_t = 1, \\ \{\text{ASK}\}, & E_t = 0. \end{cases} \tag{15}$$

This action set applies only after a Wall PASS and a Gatekeeper ASK. Actions receiving a Wall DENY or Gatekeeper BLOCK bypass the bandit and are rejected.

The eligibility indicator $E_t = 1$ requires all of the following conditions: an authenticated grant matching the current action and resource scope, a valid action binding, sufficient decision evidence, no mandatory confirmation requirement, and an ASK reason that is eligible for grant-based resolution. Otherwise, $E_t = 0$ and confirmation remains necessary.

The bandit's output is an internal approval-resolution action rather than a final authorization decision. Selecting USE_GRANT does not independently authorize execution. The controller must revalidate the grant, action arguments, resource identity, and current policy before mapping the selection to ALLOW. Failed validation prevents automatic execution.

Importantly, an ordinary user approval does not create reusable authorization. The bandit can only operate within permissions that the user has explicitly granted, and its learned parameters cannot modify the deterministic policy boundary.

The current Gatekeeper produces ALLOW, ASK, and BLOCK predictions without a separate ASK-reason classification. Reliable identification of grant-resolvable ASK decisions is therefore a proposed extension. Until such eligibility can be established, the system retains ASK as the conservative fallback.

We investigate the following contextual learning algorithms:

- **LinUCB:** Uses a linear reward model and an upper-confidence estimate to select actions. It provides an interpretable baseline for per-user contextual adaptation [8].
- **Linear Thompson Sampling:** Samples candidate decision parameters from an estimated posterior distribution. It provides an alternative approach to balancing uncertainty and exploitation.
- **Contextual ϵ-greedy:** Selects the currently preferred decision most of the time and explores alternatives with a small probability. We consider this primarily as an experimental baseline because unconstrained exploration is inappropriate for security-sensitive authorization.
- **Conservative contextual bandits:** Restrict policy updates relative to a conservative reference policy. This is a candidate direction for limiting unsafe changes during adaptation.

The bandit reward must account for more than whether a user clicked approve. It should incorporate authenticated feedback, verified execution outcomes, and the cost of unnecessary interruptions. An approval for a particular action is not an independent ground-truth label for future actions.

We will initially evaluate these algorithms using offline replay and simulated users. Live exploration that could automatically authorize actions will not be enabled until the decision policy and its safety constraints have been validated.

2) Periodic Gatekeeper Fine-Tuning: A separate learning process periodically updates the Gatekeeper's decision parameters using verified feedback. Unlike contextual bandit learning, this process modifies the model itself.

Let D_0 denote the original supervised training dataset and D_f the newly collected, independently verified feedback dataset. The updated model is trained using:

$$D_{\text{train}} = D_0 \cup D_f. \tag{16}$$

Initially, we propose freezing the encoder and fine-tuning only the decision heads. This reduces the number of trainable parameters and provides a controlled starting point for model adaptation.

The following techniques will be investigated:

- **Experience replay:** Mix original training examples with verified feedback to reduce forgetting of previously learned authorization patterns.
- **Class-balanced sampling:** Prevent the updated model from becoming biased toward frequently observed approval categories.
- **Option-order shuffling:** Randomize the ordering of ALLOW, ASK, and BLOCK options during training to reduce sensitivity to label position.

- **Regularization:** Investigate methods such as Elastic Weight Consolidation to limit undesirable changes to previously learned parameters.
- **Probability calibration:** Evaluate temperature scaling and abstention-threshold fitting on a separate calibration dataset.

Raw user approvals are not directly treated as automatic training labels. Feedback must be evaluated against the original request, current policy, action scope, and actual outcome before it enters the training dataset. Shared-model updates must also account for user privacy and prevent individual authorization preferences from being indiscriminately transferred across users.

Each candidate checkpoint is evaluated against a frozen validation suite, held-out authorization examples, and prompt-injection regression cases. A checkpoint is deployed only if it satisfies predefined evaluation criteria. Otherwise, the previous checkpoint is retained.

Preference-based approaches such as RLHF, DPO, and KTO may be considered in future work, but their direct application to Laya's discriminative decision architecture requires separate investigation.

Figure 3 distinguishes the online per-user learning process from periodic model-level adaptation.

E. Interfaces and Decision Precedence

The authorization process must distinguish model predictions from the final execution decision. We define the Wall verdict as:

$$W(P_i^{(v)}, a_t, c_t) \in \{\text{PASS}, \text{DENY}\}. \tag{17}$$

The Gatekeeper returns:

$$\mathbf{p}_t = (p_A, p_Q, p_B), \tag{18}$$

where the three entries correspond to ALLOW, ASK, and BLOCK.

Let m_t indicate whether confirmation is mandatory, e_t indicate whether critical evidence is complete, and $g_{i,t}$ indicate whether a valid explicit grant matches the proposed action.

The decision policy follows this precedence:

$$d_t = \begin{cases} \text{BLOCK}, & W_t = \text{DENY or action binding is invalid}, \\ \text{BLOCK}, & p_B \geq \tau_B, \\ \text{ASK}, & m_t = 1 \text{ or } e_t = 0, \\ \text{ALLOW}, & \hat{y}_t = \text{ALLOW and } p_A \geq \tau_A, \\ \pi_{\psi_i}(x_t), & \hat{y}_t = \text{ASK},\ g_{i,t} = 1,\ q_t = q_{\text{pref}}, \\ \text{ASK}, & \text{otherwise}. \end{cases} \tag{19}$$

Here, $\hat{y}_t = \arg\max_y p_\theta(y \mid x_t)$, and q_t denotes the reason associated with an ASK decision. The value q_{pref} identifies uncertainty that concerns reusable user authorization rather than task ambiguity or missing evidence.

The reason classification is necessary because a grant establishes permission within a defined scope; it does not establish that an action is relevant to the current task. If the implementation cannot distinguish preference-related ASK decisions from task or evidence uncertainty, the bandit must remain disabled for those decisions.

Algorithm 1 Runtime Authorization of a Proposed Tool Action

Require: User request r_i, action a_t, context c_t, policy $P_i^{(v)}$
Require: Gatekeeper f_θ, grant memory, user policy π_{ψ_i}
Ensure: Final authorization decision d_t

if action schema or binding is invalid **then**
  return BLOCK
end if
if $W(P_i^{(v)}, a_t, c_t) = \text{DENY}$ **then**
  return BLOCK
end if
$p_t \leftarrow f_\theta(r_i, a_t, c_t, P_i^{(v)})$
$\hat{y}_t \leftarrow \arg\max_y p_t(y)$
if $p_t(\text{BLOCK}) \geq \tau_B$ **then**
  return BLOCK
end if
if $m_t = 1$ **or** critical evidence is incomplete **then**
  return ASK
end if
if $\hat{y}_t = \text{ALLOW}$ **and** $p_t(\text{ALLOW}) \geq \tau_A$ **then**
  return ALLOW
end if
if $\hat{y}_t = \text{BLOCK}$ **or** $p_t(\text{BLOCK}) \geq \tau_B$ **then**
  return BLOCK
end if
if $q_t \neq q_{\text{pref}}$ **then**
  return ASK
end if
if no valid explicit grant matches a_t **then**
  return ASK
end if
$u_t \leftarrow \pi_{\psi_i}(x_t, \{\text{ASK}, \text{USE_GRANT}\})$
if $u_t = \text{USE_GRANT}$ **then**
  return ALLOW
else
  return ASK
end if

Before execution, the controller computes an action binding over the authenticated user, task, canonical action arguments, resource, and policy version:

$$b_t = H(i, \text{task}_i, a_t, \rho_t, P_i^{(v)}), \tag{20}$$

where H is a cryptographic hash over the canonical representation. The executor rechecks this binding and the current policy before dispatch. Any change to the relevant action or authorization state invalidates the previous decision.

F. Design Tradeoffs and Implementation Status

The architecture separates policy enforcement, model inference, and adaptive authorization because these components

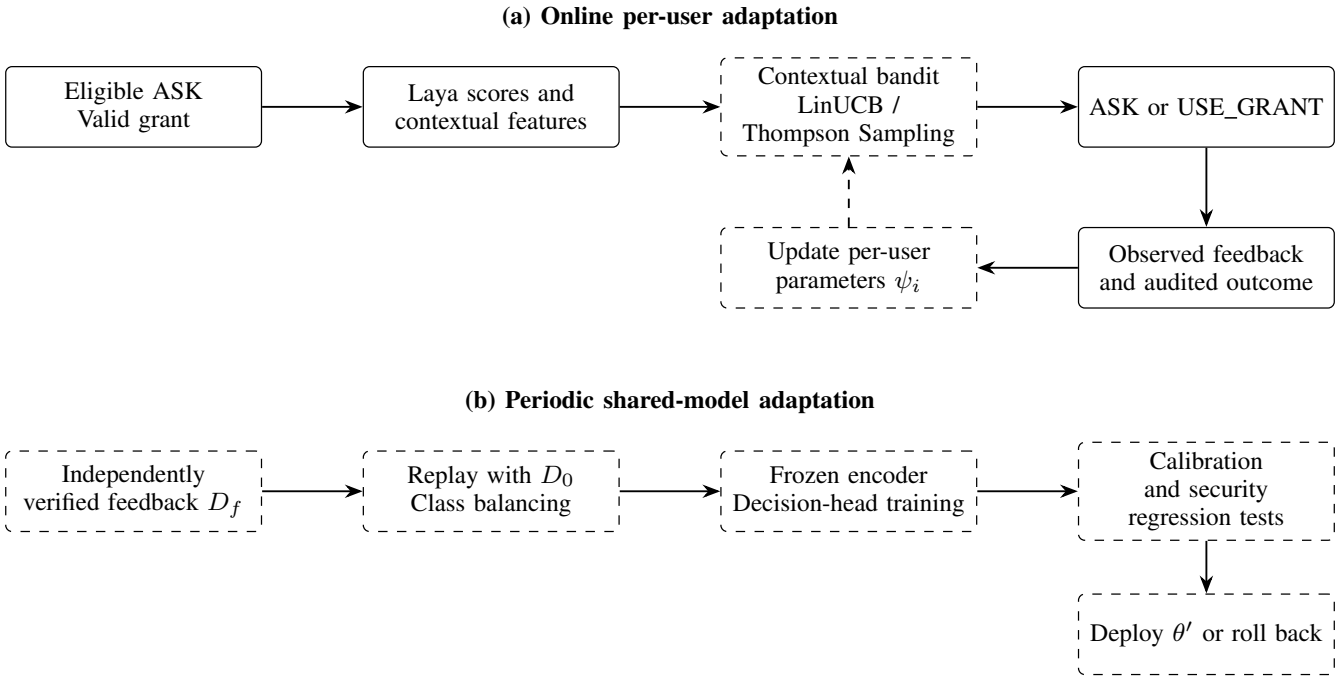


Fig. 3. Proposed adaptive authorization mechanisms. The online learner updates user-specific parameters while keeping Laya fixed. Periodic adaptation updates the shared Gatekeeper using independently verified feedback, replay, calibration, and security validation. Dashed nodes indicate research extensions.

have different update requirements. The Wall is changed only through an authenticated policy-update process. User-specific bandit parameters can be updated during interactions, whereas the shared Gatekeeper model is updated only through a separate, validated training cycle.

This separation also introduces practical tradeoffs. More restrictive grant scopes reduce the possibility of unintended permission reuse but may increase confirmation frequency. Broader trust-class grants may reduce interruptions but require stronger scope validation and revocation handling. Similarly, contextual bandit adaptation can reduce repeated confirmation requests, but its effectiveness depends on the reliability of feedback and the reward definition. Fine-tuning may improve general decision quality, but introduces risks of overfitting, forgetting, and transfer of user-specific behavior into a shared checkpoint.

The current prototype provides counting-based memory, explicit exact-action and trust-class grants, and offline permission simulations. Memory is currently maintained in RAM during a run; decision logs are not yet used to restore the learning state after a restart. The normal live approval provider also requires additional work to support grant creation. Broader trust-class grant revocation requires further validation.

The contextual bandit and periodic Laya fine-tuning described in this section are proposed extensions and have not yet been established as completed implementations. Their evaluation will determine whether they provide measurable improvements over the existing counting and explicit-grant baselines.

V. Discussion, Limitations, and Future Work

A. Security and Authorization Tradeoffs

The proposed framework separates deterministic policy enforcement from contextual authorization and feedback-driven adaptation. This separation establishes an important distinction between an action that satisfies explicit permission constraints and one that is appropriate for the user's current task. The Wall enforces the former, while the Gatekeeper evaluates the latter. Consequently, the effectiveness of the combined system depends on both the completeness of the underlying policy and the reliability of contextual decision inference.

The deterministic enforcement boundary prevents learning components from overriding explicitly prohibited actions. However, this property does not guarantee that every action permitted by the Wall is consistent with user intent. Broadly defined permissions may still admit unintended operations, while an imperfect Gatekeeper may incorrectly classify actions that satisfy the deterministic policy. The proposed architecture therefore does not eliminate the security–utility tradeoff. Instead, it provides independently enforceable constraints within which contextual decision-making can be evaluated and improved.

The use of a lightweight decision model introduces a further tradeoff between inference efficiency and authorization quality. Replacing repeated generative evaluations with structured classification may reduce computational overhead, but this advantage depends on model latency, input preparation costs, and the frequency of additional confirmation requests. A faster classifier that produces excessive false blocks or unnecessary ASK decisions may reduce overall task utility. Accordingly,

computational efficiency must be evaluated jointly with authorization accuracy and legitimate task completion rather than through model inference time alone.

B. Adaptive Authorization and Permission Creep

Approval learning introduces a separate challenge: reducing repeated user confirmations without treating previous consent as unrestricted future authority. Counting-based permission memory can reduce interruptions by automatically reusing frequently approved action patterns. However, such reuse may incorrectly generalize authorization across resources, tasks, or execution contexts.

Explicit scoped grants provide a more controlled alternative by separating approval of an individual action from authorization of future actions. Nevertheless, the security of this mechanism depends on the precision of grant matching, the handling of broader resource classes, and the correctness of expiration and revocation procedures. A valid grant must not bypass mandatory confirmation, compensate for missing task evidence, or override a Gatekeeper BLOCK.

The proposed contextual bandit introduces adaptation within these constraints. Rather than modifying the user's permission policy, it learns whether an eligible confirmation request should remain an ASK or whether an existing grant can be reused. This restricts the learner's available decisions but also limits the amount of adaptation it can provide.

An additional difficulty arises from partial feedback. User responses are directly observable primarily when confirmation is requested, while automatically authorized actions may provide only indirect execution outcomes. Furthermore, an approval does not establish that an action was objectively safe or that confirmation was unnecessary. These properties require careful reward design, conservative exploration, and evaluation using independently verified authorization outcomes.

C. Limitations and Threats to Validity

The current implementation and evaluation have several limitations.

First, the permission-classification datasets contain labels reviewed primarily through language-model-based procedures rather than independent human adjudication. Disagreements between source labels and subsequent reviews indicate that authorization criteria must be applied consistently before classifier accuracy can be interpreted as a reliable measure of decision quality.

Second, the effectiveness of the Gatekeeper depends on the information available at inference time. Context compaction addresses model input-length constraints, but it may omit evidence relevant to particular authorization decisions. Furthermore, the live controller does not yet expose all historical tool observations and trusted identity information available in offline evaluation. This creates a potential discrepancy between the model's evaluated inputs and its operational inputs.

Third, the permission-memory prototype has been evaluated primarily through offline simulations using constructed action sequences and simulated user responses. These experiments are useful for examining approval reuse and permission expansion, but they do not establish how real users would behave under repeated confirmation requests. Persistent memory restoration, complete live grant-offer integration, and revocation of overlapping broader grants also require additional implementation and validation.

Fourth, the proposed contextual bandit and periodic Gatekeeper fine-tuning mechanisms have not yet been validated as an integrated learning system. Their expected benefits remain hypotheses. In particular, model adaptation may introduce catastrophic forgetting, probability miscalibration, and unintended transfer of user-specific authorization preferences into a shared checkpoint.

Finally, the security properties of the framework assume complete mediation of tool execution, correct resource identification, authenticated approval records, and integrity of the policy enforcement components. Compromise of these trusted components or errors in the original permission policy fall outside the guarantees provided by the proposed learning mechanisms.

D. Future Research Directions

Future work will investigate three primary directions.

1) User-specific online adaptation: We plan to evaluate contextual bandit algorithms, including LinUCB, Linear Thompson Sampling, and conservative contextual exploration, for adapting confirmation decisions within existing authorization grants. The evaluation will examine confirmation reduction, unsafe grant reuse, adaptation speed, and robustness to incomplete or unreliable feedback. Offline policy evaluation will precede any deployment of learned automatic authorization behavior.

2) Continual Gatekeeper adaptation: We will investigate periodic fine-tuning of Laya's decision heads using independently verified authorization feedback. Experience replay, class-balanced sampling, option-order shuffling, and probability calibration will be evaluated to determine whether incremental training improves contextual classification without degrading previously learned behavior. Candidate checkpoints will undergo held-out authorization evaluation and prompt-injection regression testing before deployment. More extensive model adaptation will be considered only if decision-head updates prove insufficient.

3) Long-term authorization consistency: We will investigate persistent, user-isolated permission memory with authenticated grant creation, expiration, revocation, and policy-version validation. Particular attention will be given to preventing authorization transfer across unrelated tasks, resources, and users. Evaluation will also examine how policy changes and conflicting feedback affect previously learned preferences.

A broader evaluation will compare deterministic enforcement, contextual authorization, explicit grants, and adaptive learning under matched multi-step agent workloads. AgentDojo [1] provides a basis for measuring task completion and prompt-injection attack success. Additional action-level

annotations and longitudinal approval scenarios will be necessary to evaluate authorization accuracy, permission creep, and confirmation behavior directly.

VI. Conclusion

This paper presents a policy-bounded authorization architecture for tool-using LLM agents that separates deterministic permission enforcement, contextual action evaluation, and adaptive approval management. The framework evaluates proposed tool invocations throughout an agent's execution trajectory rather than relying solely on permissions established at task initiation. A deterministic Wall constrains executable actions, while a lightweight Gatekeeper provides structured ALLOW, ASK, and BLOCK assessments based on the user request and relevant execution context.

The proposed learning architecture distinguishes two forms of adaptation. User-specific contextual learning aims to reduce unnecessary confirmations within explicitly granted authorization scopes, while periodic model fine-tuning aims to improve the Gatekeeper's general decision quality using verified feedback. Neither mechanism is permitted to expand the deterministic authorization boundary independently.

The central research question is whether these mechanisms can improve authorization flexibility and computational efficiency without increasing unauthorized execution or allowing previous approvals to become unintended persistent authority. The architecture and preliminary permission-memory experiments establish a basis for investigating this question, while definitive conclusions require validated authorization labels, end-to-end benchmark evaluation, and empirical assessment of the proposed learning mechanisms.

References

[1] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://arxiv.org/abs/2406.13352.

[2] T. Shi, J. He, Z. Wang, L. Wu, H. Li, W. Guo, and D. Song, "Progent: Programmable Privilege Control for LLM Agents," arXiv:2504.11703, 2025. [Online]. Available: https://arxiv.org/abs/2504.11703.

[3] L. Tsai and E. Bagdasaryan, "Contextual Agent Security: A Policy for Every Purpose," in *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*, 2025, doi: 10.1145/3713082.3730378.

[4] J. Zhu, K. Tseng, G. Vernik, X. Huang, S. G. Patil, V. Fang, and R. A. Popa, "MiniScope: A Least Privilege Framework for Authorizing Tool Calling Agents," arXiv:2512.11147, 2025. [Online]. Available: https://arxiv.org/abs/2512.11147.

[5] R. Sequeira, S. Damianakis, U. Iqbal, and K. Psounis, "Agent-Sentry: Bounding LLM Agents via Execution Provenance," arXiv:2603.22868, 2026. [Online]. Available: https://arxiv.org/abs/2603.22868.

[6] A. Tu and M. Tu, "Task-Conditioned Least-Privilege Learning for Executable Terminal and MCP Agents," arXiv:2608.18351, 2026. [Online]. Available: https://arxiv.org/abs/2608.18351.

[7] J. Wu, Y. Nan, Y. Lin, H. Wang, Y. Xiao, S. Wang, and Z. Zheng, "SkillScope: Toward Fine-Grained Least-Privilege Enforcement for Agent Skills," arXiv:2605.05868, 2026. [Online]. Available: https://arxiv.org/abs/2605.05868.

[8] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A Contextual-Bandit Approach to Personalized News Article Recommendation," in *Proceedings of the 19th International Conference on World Wide Web*, 2010, pp. 661–670, doi: 10.1145/1772690.1772758.

[9] Y. Shu, B. Jiménez Gutiérrez, S. P. Jonnalagedda, Y. Yao, H. Sun, and Y. Su, "AgentCL: Toward Rigorous Evaluation of Continual Learning in Language Agents," arXiv:2606.02461, 2026. [Online]. Available: https://arxiv.org/abs/2606.02461.

[10] Y. Zhang and S. Li, "ConsistencyGate: Preventing Memory Contamination in LLM Agents via Self-Consistency Admission Control," arXiv:2607.22962, 2026. [Online]. Available: https://arxiv.org/abs/2607.22962.